UCS503P - Software Engineering Laboratory

CareBridge

A Structured Nurse Handoff and Patient Status Management System

Submitted by:

1024170251 Arnav Sharma
1024170243 Dhruv Dwivedi
1024170238 Pratham Nandwani
1024170225 Jalaj Poonia

Group: 3Q24 BE Third Year, CSE

Submitted to:

Anushka

Mid-Semester Evaluation
Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
- 1.1 Background and Context . . . 3
  - 1.1.1 Problem Statement . . . 3
- 1.2 Project Objectives . . . 4
  - 1.2.1 Scope . . . 4

2 Methodology and Design **5**
- 2.1 System Architecture . . . 5
  - 2.1.1 Identity . . . 5
  - 2.1.2 Data Model . . . 5
  - 2.1.3 Rules Enforced by the Database . . . 6
  - 2.1.4 Technical Stack . . . 6
- 2.2 Implementation Details . . . 7
  - 2.2.1 Module 1: Identity and Shift Lifecycle . . . 7
  - 2.2.2 Module 2: The Patient Record . . . 8
  - 2.2.3 Module 3: Per-Patient Handoff . . . 8
  - 2.2.4 Module 4: Frontend . . . 8

3 Results and Evaluation **13**
- 3.1 Testing Methodology . . . 13
- 3.2 Verification Results . . . 14
- 3.3 Evaluation Metric: Time-to-Acknowledgement . . . 14
- 3.4 Analysis . . . 15
  - 3.4.1 Challenges . . . 15
  - 3.4.2 What Is Deferred . . . 16

4 Conclusion **17**
- 4.1 Summary of Work . . . 17
- 4.2 Key Findings . . . 17
- 4.3 Future Work and Recommendations . . . 17
- 4.4 Final Remarks . . . 18

Chapter 1

Introduction

1.1 Background and Context

A nursing shift handoff is when one nurse passes responsibility for a patient to another. It happens several times a day on every ward, and it's one of the few points in clinical care where the full picture of a patient's condition lives only in someone's memory. Whatever the outgoing nurse remembers to mention is whatever the incoming nurse ends up knowing.

This isn't a hypothetical concern. The WHO lists communication during handovers as a priority patient-safety issue, and reviews of the nursing handoff literature describe the process as unstandardised and inconsistent from one nurse to the next. Accreditation bodies have issued specific guidance on poor hand-off communication, and wards that introduced structured handoff protocols saw a measurable drop in preventable adverse events.

CareBridge gives nurses a structured place to log patient status through the shift, and a way to hand that record over to the next nurse with some accountability attached.

1.1.1 Problem Statement

Looking at how handoffs actually happen on the ward, five problems kept coming up:

1. Nothing gets written down consistently, so the handoff depends entirely on what the outgoing nurse happens to remember at the time.

2. The quality of a handoff depends on who's giving it and how tired they are. A rushed handoff at the end of a long shift can easily drop the one detail that mattered most.

3. There's no record of who said what, so later there's no way to check what was actually communicated, or whether it was even acknowledged.

4. There's no way to push back on a handoff. If it's unclear, or sent to the wrong person, a nurse has no formal way to flag that.

5. None of this is measurable. A verbal handoff leaves no trace, so there's no way to tell if the process is getting better or worse over time.

Underneath all five is the same root cause: the handoff doesn't produce anything. Nothing written down means there's nothing to review, dispute, audit or measure

afterwards. CareBridge's main job is to turn the handoff into an actual record, and build accountability on top of that record.

1.2 Project Objectives

1. **A structured patient record.** Let a nurse build up notes on a patient over the course of a shift, split into *Details* and *Major Decisions*, so there's a record to hand over at the end instead of just memory.

2. **Per-patient, addressed handoffs.** Let her pick which patients to hand over, edit the text for each one, and send each to a specific, named nurse.

3. **Real accountability.** Every handoff has to be answered with *Accept*, *Decline* or *Report Issue*, with a reason required for the latter two, and every step gets written to an append-only audit trail.

4. **A measurable handoff.** Time-to-Acknowledgement (TTA) – the gap between a handoff being sent and being answered – should come straight out of stored timestamps rather than needing to be estimated.

5. **Access control that's actually verified.** A nurse should only be able to reach her own patients, or a patient currently being handed to her, and this needs to be backed by automated tests rather than just assumed to work.

1.2.1 Scope

CareBridge is designed around three actors: nurse, doctor and administrator. For this deliverable we chose to build the **nurse workflow all the way through** and leave the other two for later, because one actor that fully works is something you can actually demonstrate and test, whereas three actors that are all half-built aren't. The schema and the access-control layer were already designed with all three roles in mind, so adding the remaining interfaces later shouldn't need any structural changes. Section 3.4.2 lists what's been left out for now.

Chapter 2

Methodology and Design

2.1 System Architecture

CareBridge is a three-tier application sitting on top of a managed database.

- **Client:** a React single-page app running inside a Tauri shell. It doesn't contain any business logic of its own.
- **Application:** a FastAPI service that owns every rule – who can read which patient, what counts as a valid response, when a shift is allowed to close. It's the only component that talks to the database.
- **Data:** Supabase-managed PostgreSQL, with Row Level Security sitting underneath the application's own checks as a second layer.

The client never talks to the database directly, even though Supabase would technically let it. Routing everything through FastAPI keeps the clinical rules in one place we can audit, and keeps the database's secret key off client devices entirely.

2.1.1 Identity

Identity comes purely from the Supabase Auth JWT – nothing else. The client signs in, an interceptor attaches the token to every outgoing request, and the backend verifies that token and resolves it to a `staff` row.

No endpoint accepts a nurse ID in the request body. If one did, any caller could just claim to be any nurse, and the audit trail would end up recording whatever the client felt like claiming, rather than who actually did what.

The frontend only ever holds the *publishable* key, which is enough to sign in. The backend holds the *secret* key, which lets it bypass RLS – which is exactly why it has to do its own authorisation checks, in `services/access.py`, applied as a dependency on every route.

2.1.2 Data Model

Two design choices here are worth calling out. Notes live in a single table rather than being split into separate "details" and "decisions" tables: section is just a column, so the sectioned views and the chronological log read from the same rows and can never drift apart from each other. The second choice is that a handoff belongs to a patient, not to a whole shift. That's what lets a nurse decline the handoff for one patient without having to reject the entire batch she was sent.

Table	Purpose
`staff`	All personnel, keyed to `auth.users`, with a nurse/doctor/admin role.
`patients`	Admitted patients, each with an assigned nurse and doctor. The assigned nurse is what gives "my patients" any meaning.
`shifts`	Shift lifecycle per nurse.
`patient_notes`	The clinical record itself: body text, a section (`detail` or `major_decision`), an author and a shift.
`handoffs`	One row per patient handed over – sender, recipient, text, response, and the timestamps TTA is derived from.
`handoff_events`	Append-only audit trail: `sent`, `viewed`, `accepted`, `declined`, `issue_reported`.
`alerts`	Nurse-to-doctor escalation, wired up alongside the doctor interface.

Table 2.1: Database schema

2.1.3 Rules Enforced by the Database

A few invariants matter enough that they're enforced as database constraints rather than left to application code, so they hold no matter which code path runs:

- A handoff can't be sent to yourself.
- Declining, or reporting an issue, requires a non-empty reason – neither can be submitted with a blank box.
- A resolved handoff always has a `responded_at` set, and a pending one never does, which keeps TTA well-defined without any extra logic.
- Only one *pending* handoff is allowed per patient at a time, and only one *active* shift per nurse, both enforced with partial unique indexes.

2.1.4 Technical Stack

- **Frontend:** React 19, TypeScript, Vite, React Router, Tailwind CSS, packaged in Tauri v2.
- **Backend:** Python 3.13, FastAPI, Uvicorn, Pydantic v2.
- **Database and auth:** Supabase (PostgreSQL 17) with RLS enabled; Supabase Auth handles identity.

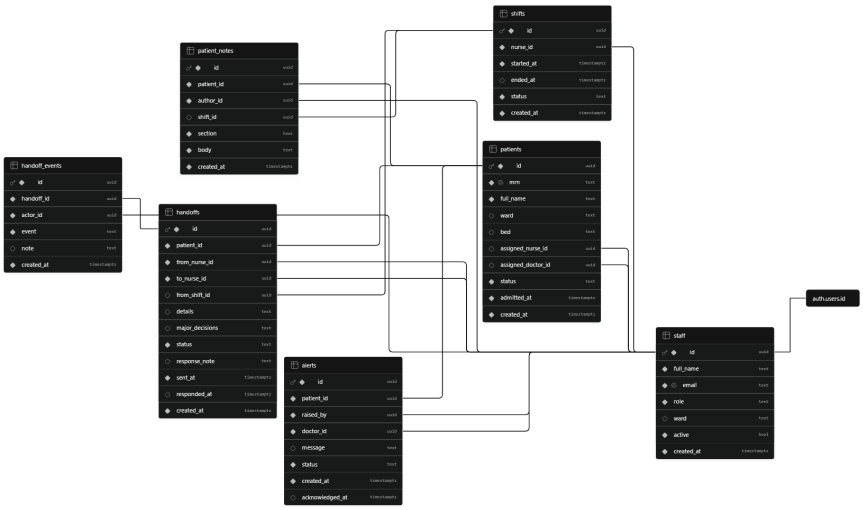


Figure 2.1: Database Schema (Entity-Relationship Diagram)

- **Testing:** pytest, using FastAPI's `TestClient`.

2.2 Implementation Details

2.2.1 Module 1: Identity and Shift Lifecycle

The backend verifies each bearer token and resolves it to a `staff` row, caching verified tokens briefly so every single request doesn't need its own round trip to the auth server.

There are two separate authorisation checks, kept deliberately apart: `ensure_can_view_patient`, which is also true for a patient currently being handed to a nurse (she needs to read the record before she can decide whether to accept it), and `ensure_assigned_patient`, which is only true once the patient is actually hers. Merging these into one check would either stop the receiving nurse from reading what she's being asked to accept, or let her start editing a patient who isn't hers yet.

Shift handling is idempotent: `GET /api/shifts/current` either returns the currently open shift or opens a new one, so refreshing the page can't accidentally create duplicate shifts. Trying to close a shift while patients are still assigned with no handoff sent gets rejected with HTTP 409.

2.2.2 Module 2: The Patient Record

Each patient's screen shows three views over the same underlying rows: *Details*, *Major Decisions*, and a full chronological *Log*. Notes build up over the shift, which turns the handoff into something the nurse reviews and edits, rather than something she has to recall from scratch.

Notes are append-only – there's no update or delete path anywhere in the API or the RLS policies. A note that could be silently edited afterwards wouldn't really support the accountability the system is supposed to provide.

2.2.3 Module 3: Per-Patient Handoff

The flow has three stages: **prepare**, **review**, **respond**.

Preparing a handoff pulls together the shift's notes for each selected patient and pre-fills a *Details* and *Major Decisions* box by grouping the notes by section. Since it's the nurse's own text, this just saves her retyping it, and nothing is written to the database at this point. Reviewing lets her edit either box and choose a named recipient. Sending creates one `handoffs` row per patient, with `sent_at` set and a `sent` event logged.

The receiving nurse opens the handoff, which logs a `viewed` event, so "she read it before responding" is something the system can show rather than assume, and she reads the Details, Major Decisions and Log before deciding how to respond.

The three possible responses aren't just three flavours of the same action:

- **Accept** transfers the patient to her.
- **Decline** requires a reason and leaves the patient with the sender – used when it's the wrong recipient, or too much information is missing to act on.
- **Report Issue** also requires a reason, but still transfers the patient. Something about the handoff is wrong and needs sorting out, but the patient can't be left without a nurse while that happens.

Every response sets `responded_at` and appends an event. The update only goes through if the row is still pending, so if two responses somehow arrive at once, the second one gets an HTTP 409 instead of silently overwriting the first nurse's decision.

2.2.4 Module 4: Frontend

There are three main surfaces – *Shift*, *My Patients* and *Handoff* – with nested routes for an individual patient, a note composer, and a handoff review screen. Everything except the login page sits behind a `RequireAuth` guard.

The visual design is kept deliberately simple: one hue at varying opacity, with colour reserved for the states that matter – amber for pending, emerald for accepted, rose for declined. The layout is built for shared ward tablets, using responsive units and safe-area insets so it holds up across whatever screen size a given ward happens to have.

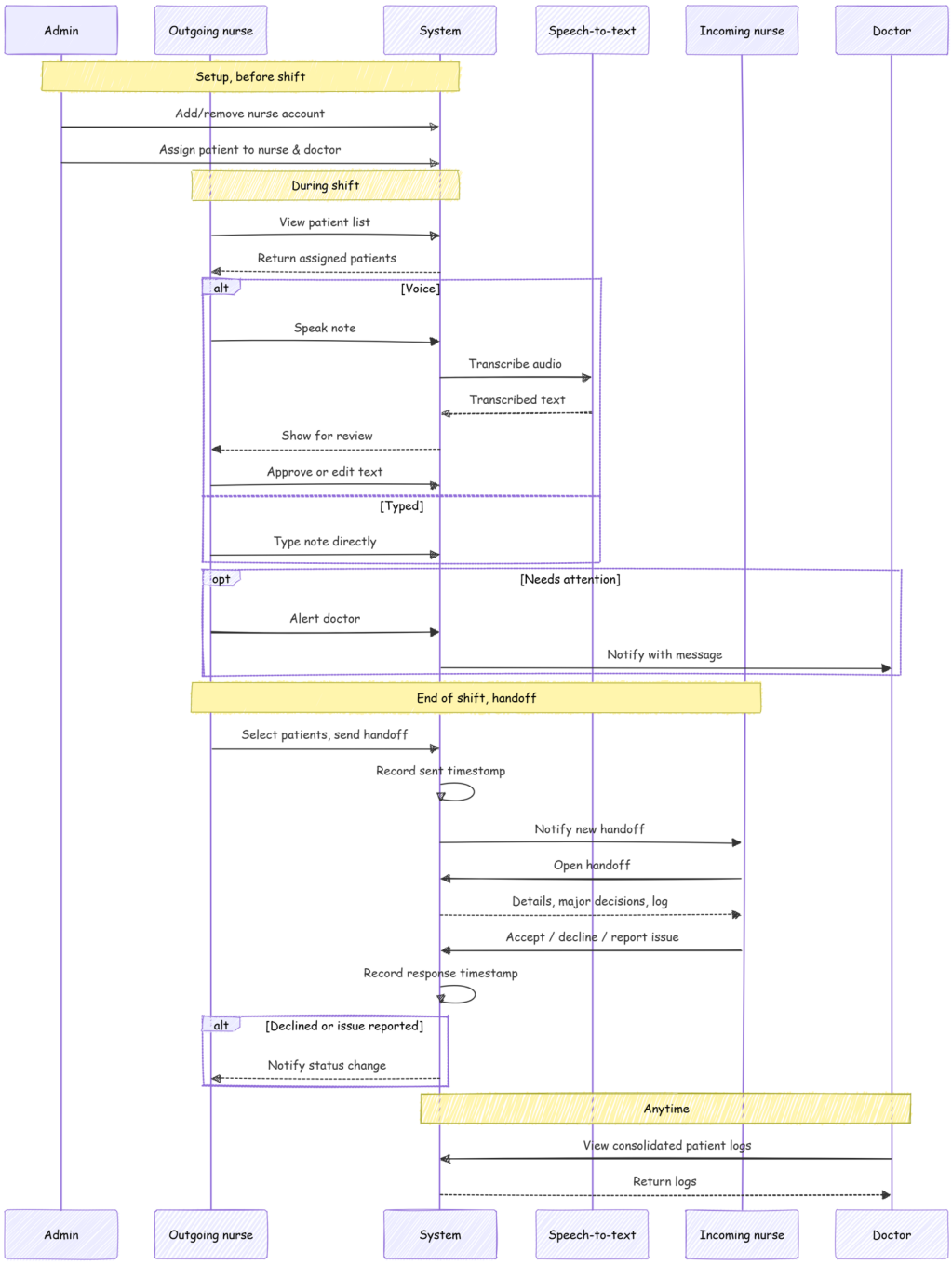


Figure 2.2: Handoff Sequence Diagram

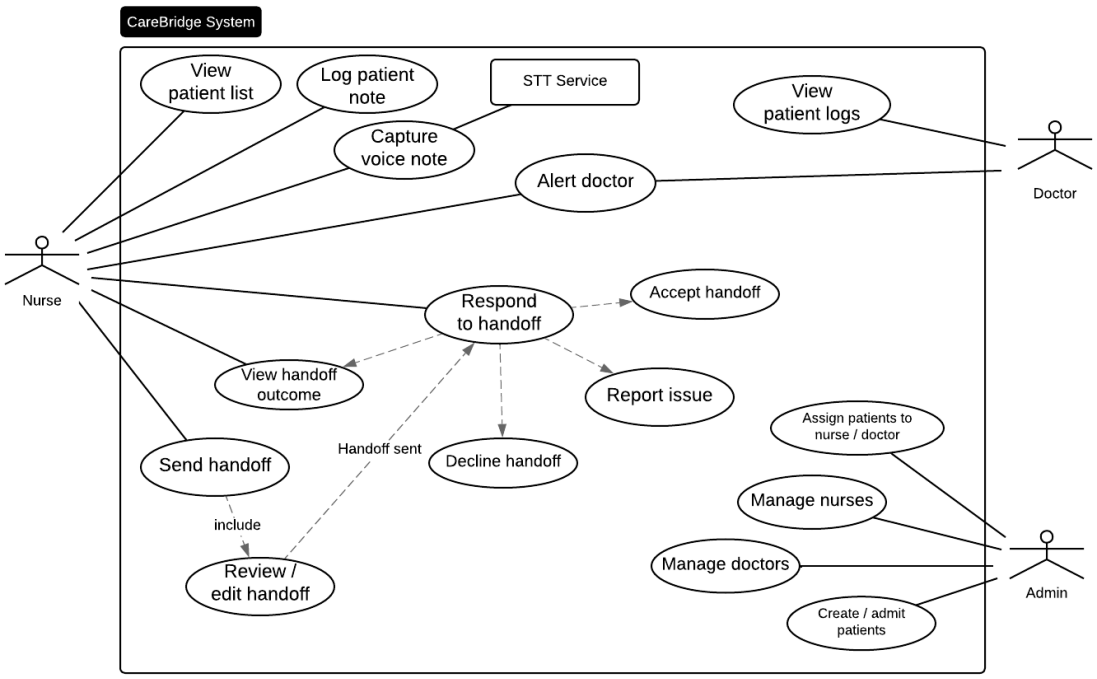


Figure 2.3: Use Case Diagram

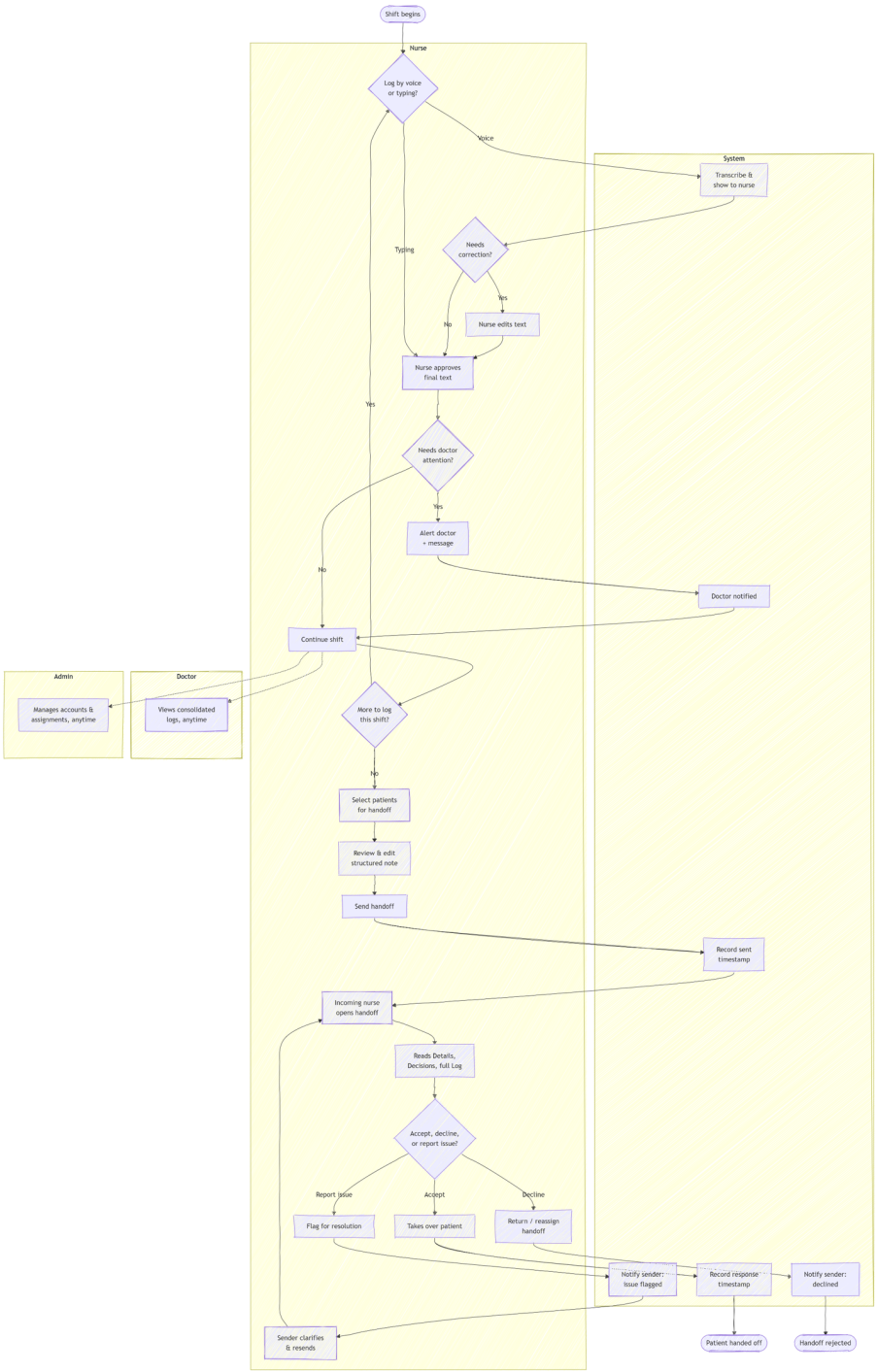


Figure 2.4: Handoff Activity Diagram

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing was done at four different levels, because a passing test at one level doesn't say much about the others. A database constraint being enforced doesn't tell you the API route that hits it behaves correctly, and a route returning the right status code doesn't tell you it wrote the right thing to the database.

- **Route-level tests.** A `pytest` suite exercises the handoff state machine and note recording through FastAPI's `TestClient`. Instead of hitting a real database, the identity dependency is overridden with a fake logged-in nurse, and the data layer is swapped for a recording double that remembers every write it's asked to make. Each test then checks the actual writes that were attempted – for example, that declining a handoff really does leave the patient assigned to the sender, not just that the API returned a 200. A status code on its own can't tell you that.

- **Constraint tests.** Every database constraint listed in Section 2.1.4 was tested by directly attempting the forbidden operation against the real schema, inside a transaction that gets rolled back afterwards, and checking that PostgreSQL rejects it.

- **Integration checks.** The schema, foreign keys and seed data were checked against the live Supabase instance itself, including signing in as one of the seeded nurses and confirming she resolves to exactly the patients she's supposed to see, and no others.

- **Build and security checks.** The frontend build type-checks the entire client before it bundles anything, and Supabase's built-in security advisor was run after every migration to catch any table left without an RLS policy.

In short: the route tests check that the business logic behaves correctly when everything below it is trusted, the constraint tests check that the database itself refuses bad data even if a bug slips past the application layer, the integration checks confirm the whole thing actually wires together correctly against the real database rather than a mock, and the build/security checks catch the two classes of mistake – a broken frontend build, or a table left unprotected – that none of the other three would notice.

3.2 Verification Results

Check	Count	Result
Automated backend tests	15	All pass
Handoff state machine and access	10	All pass
Note recording and access	5	All pass
Database constraint checks	7	All enforced
Tables with Row Level Security enabled	7 of 7	Enabled
Critical RLS security findings	0	Clear
Frontend type-check and build	1	Passes

Table 3.1: Verification results

Running the constraint checks against the live schema confirmed all of the following: a handoff addressed to yourself gets rejected; declining or reporting an issue without a reason gets rejected; sending a second pending handoff for a patient who already has one gets rejected; resolving a handoff without setting a response timestamp gets rejected; accepting a handoff correctly starts the TTA clock; and a patient becomes eligible for a new handoff again as soon as the previous one is resolved.

Access control was checked at both layers independently. At the database layer, querying as a seeded nurse returned exactly the 3 patients (out of 6 seeded overall) she's assigned to, and no others. At the application layer, tests confirm that a nurse can't write notes for another nurse's patient, can't read a handoff addressed to someone else, and that a sender can't respond to her own handoff on the recipient's behalf.

RLS is switched on with policies covering all seven tables, and the anonymous role's access has been revoked entirely, so nothing in the database is reachable, or even discoverable, without a valid signed-in session.

Across the five routers (staff, shifts, patients, notes, handoff) the API currently exposes 16 endpoints.

3.3 Evaluation Metric: Time-to-Acknowledgement

The main metric the system is designed around is **Time-to-Acknowledgement**: how long it takes between a handoff being sent and being answered. Since every handoff row stores both `sent_at` and `responded_at`, and a constraint guarantees the pair stays consistent, the metric can be pulled directly with a simple query:

```
SELECT avg(responded_at - sent_at)
FROM   handoffs
WHERE  responded_at IS NOT NULL;
```

A secondary metric, clean-accept rate – the share of handoffs accepted without an issue being reported – comes from the same table with no extra work.

We're not reporting any actual TTA numbers here. That would need a pilot with nurses who actually work on a ward, which hasn't happened yet. Numbers generated by our own team clicking through our own interface would mostly measure how familiar we are with the app we built, not how handoffs perform in practice. At this stage, what we're delivering is the instrumentation needed to measure it later, not the measurement itself.

3.4 Analysis

All five objectives from Section 1.2 are implemented and backed up by the tests above. Accountability runs end to end: identity always comes from a verified token, every state change is recorded along with who made it, and the `viewed` event lets the system tell the difference between a handoff that was actually read and one that was accepted without being looked at properly. Measurability comes from the schema itself rather than from anyone remembering to log something correctly. Access control is enforced at two independent layers, and the tests check the actual writes made, not just the response codes returned.

3.4.1 Challenges

Concurrent database access. Each screen fires off several API requests at once, and each of those requests runs several queries in turn. The synchronous database client we started with shares one connection pool across a thread pool, and under load this failed intermittently on Windows. Because the failure took down every in-flight request on that connection, a whole screen could fail at once instead of just one request. Switching to the asynchronous client got rid of the shared-pool problem: across a repeated 175-request load test, failures went from 19 down to zero.

A permissions bug that looked like a data bug. Once RLS was turned on, backend reads started failing with *permission denied*, but the symptom that showed up on screen was just a nurse appearing to have no patients at all, which pointed us in the wrong direction at first. The actual cause was that the backend had been configured with the publishable key instead of the secret key, and the publishable key can't bypass RLS. Beyond fixing the key, we added a startup check that now refuses to launch at all if it detects a publishable key, so this bug can't come back disguised as an empty patient list again.

Defining what "Report Issue" should actually do. Accept and Decline are fairly self-explanatory, but Report Issue took real discussion. Making it behave like Decline risks bouncing a patient back to a sender who might not even be on shift anymore. Making it behave like Accept risks the underlying problem just getting

forgotten once the patient changes hands. We settled on transferring care while keeping the issue on record, on the reasoning that patient coverage should never be the thing that lapses while a documentation problem gets sorted out.

3.4.2 What Is Deferred

- The **doctor interface** – consolidated per-patient logs and nurse-to-doctor escalation. The `alerts` table and `assigned_doctor_id` column already exist to support this.
- The **administrator interface** – staff management and patient assignment, which right now can only be done through direct database access.
- **Voice capture** for dictating notes instead of typing them.
- **Push notifications** for new or disputed handoffs – right now the inbox badge only refreshes when you navigate to it.
- **Deployment** – the backend isn't hosted anywhere yet.
- **The pilot**, and any measured outcome data that would come out of it.

Chapter 4

Conclusion

4.1 Summary of Work

CareBridge delivers a working nurse handoff workflow, start to finish. A nurse signs in against a real identity provider, sees the patients currently under her care, and builds up a structured record for each one across Details and Major Decisions as the shift goes on. At the end of her shift she picks which patients to hand over, reviews and edits the text for each, and sends them to a named colleague, who reads the full record and responds with Accept, Decline or Report Issue, giving a reason wherever one is required. Every step along the way is recorded with who did it and when.

All five objectives are met, backed up by 15 automated tests, 7 constraint checks, RLS policies on all seven tables, and a clean security advisory report.

4.2 Key Findings

- Accountability only works once there's an actual object to attach it to. A handoff can't be declined, disputed or measured until it exists as a durable record with a named sender and recipient. Making handoffs per-patient rather than per-shift is what let the three response types work coherently together.

- Rules are more reliable when the database enforces them. A rule in the interface only holds until someone builds a second way of reaching the same data, while a rule in the database holds regardless of how the data is reached.

- Measurement has to be planned for from the start. TTA works cleanly because the two timestamps and the matching constraint were part of the schema from day one; bolting a metric like this on afterwards would have been a lot harder.

4.3 Future Work and Recommendations

1. Build out the remaining actors – the doctor and administrator screens. The schema already supports both, so this shouldn't need any migrations.

2. Run an actual pilot and measure TTA, ideally with 10–15 nurses or a simulated ward, and establish a baseline for the current (non-CareBridge) process by observation first, so the comparison means something rather than just comparing CareBridge to nothing at all.

3. Add voice capture for entering notes, with a mandatory review step so nothing transcribed automatically reaches the record without the nurse checking it first.

4. Deploy the backend to managed hosting, with CORS locked down to the deployed client's origin only.

5. Add push notifications so a nurse finds out about a handoff without having to go and check for it manually.

6. Explore AI-assisted features on top of the existing note data: automatic summaries of a patient's accumulated shift notes to speed up handoff review, and AI-based categorisation that suggests whether a new note belongs under Details or Major Decisions. Either would sit alongside the nurse's own judgement as a suggestion, not a replacement for it – she would still confirm or edit before anything is saved.

4.4 Final Remarks

The biggest lesson from this project was about where to put a rule. A rule in the interface holds only until someone builds a second path to the same data; a rule in the database holds no matter what. Working through that repeatedly shifted the question we were asking, from "how do we get the app to do the right thing" to "how do we make the wrong thing impossible in the first place".

Software isn't going to make handoffs easy. But the specific failure this project set out to fix – information that doesn't survive the handover from one person to the next – is now recorded, addressed to someone, answerable, and measurable.